

Univerzitet u Kragujevcu
Fakultet inženjerskih nauka
Predmet: Kriptografija

Projektni zadatak za školsku 2025/26. godinu

Studenti rade jedan od ponuđenih zadataka. Broj projektnog zadatka koji se radi se računa po formuli $\text{broj_zadatka} = (\text{broj_indeksa} + \text{godina_upisa}) \bmod 10$.

Projekat je potrebno završiti i okačiti najkasnije dan uoči termina odbrane projekta u nekom roku.

Potrebno je koristiti Github za menadžment koda projekta, uz dobar Readme opis koji će biti vidljiv budućim poslodavcima kada se u CV stavi Github nalog studenta. Github nalog profesora je milancabarkapa, pa ga možete pronaći na Github-u i dodati na projekat.

Za sve zadatke (0–9) treba:

- koristiti MetaMask i naloge na njemu za potpisivanje transakcija i poruka,
- koristiti Sepolia testnu mrežu za deployment i testiranje ugovora,
- testni ETH se dobija sa faucet-a <https://cloud.google.com/application/web3/faucet/ethereum/sepolia> (besplatni tokeni za učenje i testiranje).

Projektni zadatak 0: DApp za multisignature (M-of-N) novčanik za autorizaciju transakcija

Cilj projekta

Cilj projekta je da studenti razviju decentralizovanu aplikaciju (DApp) koja implementira multisignature (multisig) novčanik, gde se transakcija izvršava tek kada je odobri unapred definisani minimalni broj vlasnika (M od ukupno N).

U ovom zadatku je fokus na distribuiranoj kontroli i otpornosti na kompromitaciju pojedinačnog ključa.

Umesto da jedan privatni ključ ima potpunu kontrolu nad sredstvima, ovlašćenje se deli između više učesnika, čime se eliminiše jedinstvena tačka otkaza i znatno otežava zloupotreba ili krađa sredstava.

Ključne komponente projekta

- Pametni ugovor koji vodi evidenciju o vlasnicima novčanika, pragu potrebnih potpisa (M) i predloženim transakcijama, prihvata pojedinačna odobrenja i izvršava transakciju kada se dostigne prag.
- Front-end aplikacija koja omogućava vlasnicima da predlože transakciju, pregledaju predložene transakcije, daju ili povuku svoje odobrenje i prate status (koliko je potpisa prikupljeno).
- Integracija sa Web3/ethers.js za komunikaciju sa pametnim ugovorom i potpisivanje akcija preko MetaMask-a.
- Deployment i testiranje u Truffle ili Hardhat okruženju, sa testovima za predlaganje transakcije, prikupljanje odobrenja, izvršenje po dostizanju praga, odbijanje pre dostizanja praga i pokušaj dvostrukog odobrenja istog vlasnika.

Detalji pametnog ugovora

- Ugovor treba da čuva listu ovlašćenih vlasnika i prag M, koji se postavlja prilikom inicijalizacije.
- Potrebno je omogućiti predlaganje transakcije (primalac, iznos, opcioni podaci), evidenciju odobrenja po vlasniku i izvršenje tek kada broj odobrenja dostigne prag M.
- Svaki vlasnik može da odobri transakciju samo jednom; mora postojati zaštita od ponavljanja odobrenja i ponovnog izvršavanja već izvršene transakcije.
- Ugovor emituje događaje pri predlaganju transakcije, svakom odobrenju, opozivu odobrenja i konačnom izvršenju.

Dodatni kriptografski i bezbednosni zahtevi

- Autorizacija se zasniva na ECDSA potpisima koje generišu vlasnici preko svojih naloga (MetaMask); ugovor verifikuje da odobrenje dolazi isključivo od ovlašćene adrese.
- Sistem mora sprečiti reentrancy napade pri izvršenju transakcije (npr. checks-effects-interactions obrazac).
- Promena skupa vlasnika ili praga M mora i sama prolaziti kroz multisig proces, a ne biti dozvoljena pojedinačnom adresom.
- Svi zapisi (predlozi, odobrenja, izvršenja) moraju biti vremenski obeleženi i javno proverljivi.

Bodovanje

- Pametni ugovor i deployment na Sepolia mreži: 9 poena
- Web3/React integracija i front-end aplikacija: 7 poena
- Testiranje funkcionalnosti - obavezni unit testovi (Truffle/Hardhat ili Remix) uz end-to-end proveru nad ugovorom na Sepolia mreži: 4 poena

Ukupno: 20 poena (bez testiranja: maksimalno 16 poena)

Implementacija se može uraditi u Remix IDE ili u Truffle/Hardhat okruženju. Izbor alata ne utiče na maksimalan broj poena. Bez obzira na izabrani alat, ugovor mora biti deploy-ovan na Sepolia testnu mrežu, a aplikacija (Web3/React) mora biti povezana sa pametnim ugovorom upravo na Sepolia mreži. Time se stiče iskustvo u izradi kompletnih decentralizovanih aplikacija koje obuhvataju i blockchain i web sloj.

Testiranje funkcionalnosti (4 poena) je obavezno i obuhvata dva dela. Prvo, student je dužan da napiše unit testove za pametni ugovor, automatizovane testove u Truffle/Hardhat okruženju ili pomoću Solidity Unit Testing plugin-a u Remix-u koji pokrivaju ključne funkcije i granične slučajeve (validne i nevalidne scenarije, kontrolu pristupa, emitovanje događaja). Drugo, ispravan rad mora biti potvrđen i end-to-end proverom kroz samu aplikaciju (Web3/React) nad ugovorom deploy-ovanim na Sepolia mreži. Ukoliko student ne uradi unit testove, maksimalan broj poena za ceo zadatak iznosi 16 poena (9 + 7). Ovladavanje unit testovima i deployment skriptama predstavlja standardnu praksu u profesionalnom razvoju i čini studente konkurentnijim na tržištu rada.

Projektni zadatak 1: DApp za anonimnu verifikaciju atributa (zero-knowledge dokaz članstva)

Cilj projekta

Cilj projekta je da studenti razviju decentralizovanu aplikaciju (DApp) koja omogućava korisniku da dokaže da poseduje određeni atribut ili da pripada nekoj grupi (npr. da je registrovani član, da je punoletan, da poseduje ovlašćenje) bez otkrivanja svog identiteta ili samih podataka.

U ovom zadatku je fokus na privatnosti i minimizaciji otkrivanja podataka (data minimization).

Korišćenjem dokaza sa nultim znanjem (zero-knowledge proof) korisnik uverava ugovor da tvrdnja važi, dok proverilac ne saznaje ništa više od same činjenice da je tvrdnja istinita.

Ključne komponente projekta

- Pametni ugovor koji čuva javni koren (npr. Merkle root) skupa ovlašćenih članova i verifikuje dostavljeni dokaz pripadnosti, evidentirajući uspešne verifikacije bez vezivanja za identitet.
- Front-end aplikacija koja korisniku omogućava da lokalno generiše dokaz pripadnosti (Merkle proof ili ZK dokaz) i pošalje ga ugovoru na verifikaciju, uz jasan prikaz šta se dokazuje, a šta ostaje skriveno.
- Integracija sa Web3/ethers.js radi slanja dokaza i čitanja rezultata verifikacije.
- Deployment i testiranje u Truffle ili Hardhat okruženju. Testovi treba da pokriju validan dokaz člana skupa, odbijanje dokaza za nečlana, kao i pokušaj ponovne upotrebe istog dokaza.

Detalji pametnog ugovora

- Ugovor treba da čuva javni kriptografski commitment skupa ovlašćenih korisnika (npr. Merkle root) koji postavlja administrator ili ovlašćeni entitet.
- Potrebno je omogućiti verifikaciju dostavljenog dokaza pripadnosti bez čuvanja ličnih podataka korisnika na lancu.
- Sistem treba da spreči ponovnu upotrebu istog dokaza (npr. evidencijom iskorišćenih nullifier vrednosti), kako jedan korisnik ne bi mogao da neograničeno koristi isti dokaz.
- Ugovor emituje događaje pri uspešnoj i neuspešnoj verifikaciji, ali bez otkrivanja identiteta korisnika.

Dodatni kriptografski i bezbednosni zahtevi

- Dokaz pripadnosti zasniva se na Merkle stablu (uparivanje hash-eva uz precizno definisan redosled levo/desno) ili na ZK shemi; ista hash funkcija mora se dosledno koristiti pri građenju stabla i verifikaciji.
- Na blockchain se ne smeju upisivati lični podaci niti tajne vrednosti; čuvaju se isključivo javni koren i metapodaci.
- Mehanizam nullifier-a (jednokratne oznake) mora garantovati nepovezivost (unlinkability), ali sprečiti dvostruko korišćenje.
- Sistem treba da bude otporan na pokušaje falsifikovanja dokaza i da jasno razlikuje validan od nevalidnog dokaza.

Bodovanje

- Pametni ugovor i deployment na Sepolia mreži: 9 poena

- Web3/React integracija i front-end aplikacija: 7 poena
- Testiranje funkcionalnosti - obavezni unit testovi (Truffle/Hardhat ili Remix) uz end-to-end proveru nad ugovorom na Sepolia mreži: 4 poena

Ukupno: 20 poena (bez testiranja: maksimalno 16 poena)

Implementacija se može uraditi u Remix IDE ili u Truffle/Hardhat okruženju. Izbor alata ne utiče na maksimalan broj poena. Bez obzira na izabrani alat, ugovor mora biti deploy-ovan na Sepolia testnu mrežu, a aplikacija (Web3/React) mora biti povezana sa pametnim ugovorom upravo na Sepolia mreži. Time se stiče iskustvo u izradi kompletnih decentralizovanih aplikacija koje obuhvataju i blockchain i web sloj.

Testiranje funkcionalnosti (4 poena) je obavezno i obuhvata dva dela. Prvo, student je dužan da napiše unit testove za pametni ugovor, automatizovane testove u Truffle/Hardhat okruženju ili pomoću Solidity Unit Testing plugin-a u Remix-u koji pokrivaju ključne funkcije i granične slučajeve (validne i nevalidne scenarije, kontrolu pristupa, emitovanje događaja). Drugo, ispravan rad mora biti potvrđen i end-to-end proverom kroz samu aplikaciju (Web3/React) nad ugovorom deploy-ovanim na Sepolia mreži. Ukoliko student ne uradi unit testove, maksimalan broj poena za ceo zadatak iznosi 16 poena (9 + 7). Ovladavanje unit testovima i deployment skriptama predstavlja standardnu praksu u profesionalnom razvoju i čini studente konkurentnijim na tržištu rada.

Projektni zadatak 2: DApp za aukciju zapečaćenih ponuda (commit–reveal sealed-bid auction)

Cilj projekta

Cilj projekta je da studenti razviju decentralizovanu aplikaciju (DApp) koja sprovodi aukciju sa zapečaćenim ponudama, gde učesnici prvo dostavljaju skrivene (zapečaćene) ponude, a tek kasnije ih otkrivaju, čime se sprečava da bilo ko vidi tuđu ponudu pre kraja faze licitiranja.

U ovom zadatku je fokus na poverljivosti ponuda i poštenom ishodu aukcije.

Commit–reveal shema obezbeđuje da nijedan učesnik, pa ni organizator, ne može da prilagodi svoju ponudu na osnovu tuđih, čime se postiže nepristrasnost i otpornost na manipulaciju.

Ključne komponente projekta

- Pametni ugovor koji upravlja fazama aukcije (commit faza, reveal faza, određivanje pobjednika), prihvata zapečaćene ponude i kasnije proverava njihovu validnost pri otkrivanju.
- Front-end aplikacija koja omogućava učesniku da formira i pošalje zapečaćenu ponudu, kasnije je otkrije (vrednost + tajni broj), i na kraju vidi pobjednika i ishod.
- Integracija sa Web3/ethers.js radi interakcije sa ugovorom i potpisivanja transakcija preko MetaMask-a.
- Deployment i testiranje u Truffle ili Hardhat okruženju. Testovi treba da pokriju slanje zapečaćene ponude, validno otkrivanje, pokušaj otkrivanja ponude koja se ne poklapa sa commitmentom, kao i određivanje pobjednika i povraćaj depozita gubitnicima.

Detalji pametnog ugovora

- Aukcija se odvija u fazama: faza slanja zapečaćenih ponuda (commit), faza otkrivanja (reveal) i faza zaključenja sa proglašenjem pobjednika.
- U commit fazi učesnici upisuju samo hash kombinacije ponude i tajnog broja; u reveal fazi otkrivaju stvarnu vrednost i tajni broj radi provere podudaranja sa zabeleženim hash-om.
- Ugovor mora sprečiti izmenu ponude nakon slanja i obezbediti da se samo validno otkrivene ponude uzimaju u obzir.
- Ugovor emituje događaje pri slanju ponude, otkrivanju, proglašenju pobjednika i povraćaju sredstava neuspešnim učesnicima.

Dodatni kriptografski i bezbednosni zahtevi

- Tajnost ponuda postiže se commit–reveal shemom: na lanac se prvo upisuje hash (npr. keccak256) kombinacije iznosa i nasumičnog tajnog broja, a stvarna vrednost se otkriva tek u reveal fazi.
- Tajni broj (salt) mora biti dovoljne entropije kako bi se sprečio brute-force pogađanje ponude iz hash vrednosti.
- Trajanje faza mora se meriti pomoću vremenskih oznaka bloka (block timestamp), uz jasne granice prelaska iz faze u fazu.
- Potrebno je obezbediti zaštitu depozita učesnika i sprečiti zloupotrebu (npr. neotkrivanje ponude), kao i reentrancy zaštitu pri povraćaju sredstava.

Bodovanje

- Pametni ugovor i deployment na Sepolia mreži: 9 poena

- Web3/React integracija i front-end aplikacija: 7 poena
- Testiranje funkcionalnosti - obavezni unit testovi (Truffle/Hardhat ili Remix) uz end-to-end proveru nad ugovorom na Sepolia mreži: 4 poena

Ukupno: 20 poena (bez testiranja: maksimalno 16 poena)

Implementacija se može uraditi u Remix IDE ili u Truffle/Hardhat okruženju. Izbor alata ne utiče na maksimalan broj poena. Bez obzira na izabrani alat, ugovor mora biti deploy-ovan na Sepolia testnu mrežu, a aplikacija (Web3/React) mora biti povezana sa pametnim ugovorom upravo na Sepolia mreži. Time se stiče iskustvo u izradi kompletnih decentralizovanih aplikacija koje obuhvataju i blockchain i web sloj.

Testiranje funkcionalnosti (4 poena) je obavezno i obuhvata dva dela. Prvo, student je dužan da napiše unit testove za pametni ugovor, automatizovane testove u Truffle/Hardhat okruženju ili pomoću Solidity Unit Testing plugin-a u Remix-u koji pokrivaju ključne funkcije i granične slučajeve (validne i nevalidne scenarije, kontrolu pristupa, emitovanje događaja). Drugo, ispravan rad mora biti potvrđen i end-to-end proverom kroz samu aplikaciju (Web3/React) nad ugovorom deploy-ovanim na Sepolia mreži. Ukoliko student ne uradi unit testove, maksimalan broj poena za ceo zadatak iznosi 16 poena (9 + 7). Ovladavanje unit testovima i deployment skriptama predstavlja standardnu praksu u profesionalnom razvoju i čini studente konkurentnijim na tržištu rada.

Projektni zadatak 3: DApp za decentralizovani žreb sa proverljivom slučajnošću (verifiable randomness)

Cilj projekta

Cilj projekta je da studenti razviju decentralizovanu aplikaciju (DApp) koja sprovodi pošten i proverljiv žreb (lutriju / izbor dobitnika) korišćenjem mehanizma proverljive slučajnosti, tako da nijedan učesnik ni organizator ne može unapred da predvidi ili namesti rezultat.

U ovom zadatku je fokus na poštenoj i proverljivoj slučajnosti (fairness).

Pošto je blockchain deterministički i javan, naivno generisanje slučajnih brojeva je nebezbedno; zato se koristi commit–reveal doprinos više učesnika (ili VRF pristup) kako bi krajnji rezultat bio nepredvidiv i kasnije proverljiv.

Ključne komponente projekta

- Pametni ugovor koji prikuplja doprinose slučajnosti od učesnika, kombinuje ih u finalni nasumičan ishod i bira dobitnika na proverljiv način.
- Front-end aplikacija koja omogućava registraciju učesnika, slanje commit-a (hash tajne vrednosti), kasnije otkrivanje vrednosti i prikaz konačnog rezultata žreba.
- Integracija sa Web3/ethers.js radi komunikacije sa ugovorom.
- Deployment i testiranje u Truffle ili Hardhat okruženju. Testovi treba da pokriju ceo ciklus: prikupljanje commit-a, otkrivanje, izračunavanje finalne slučajne vrednosti, izbor dobitnika i odbijanje učesnika koji nisu otkrili svoju vrednost.

Detalji pametnog ugovora

- Ugovor treba da vodi evidenciju o registrovanim učesnicima i njihovim commit vrednostima (hash doprinosa slučajnosti).
- U fazi otkrivanja učesnici otkrivaju svoje vrednosti, a ugovor proverava podudaranje sa prethodno upisanim hash-om.
- Finalna slučajna vrednost izračunava se kombinovanjem svih validno otkrivenih doprinosa (npr. keccak256 konkatencije), čime nijedan pojedinac ne kontroliše ishod.
- Ugovor emituje događaje pri registraciji, commit-u, otkrivanju i objavi dobitnika.

Dodatni kriptografski i bezbednosni zahtevi

- Slučajnost se obezbeđuje commit–reveal shemom uz doprinose više nezavisnih učesnika; alternativno se može opisati i upotreba VRF (Verifiable Random Function) pristupa.
- Ugovor se ne sme oslanjati na block.timestamp, blockhash ili druge predvidljive/manipulabilne izvore kao jedini izvor slučajnosti.
- Mora se obezbediti otpornost na strategiju neotkrivanja vrednosti (npr. kažnjavanje učesnika koji ne otkriju doprinos ili njihovo izuzimanje).
- Konačni ishod mora biti javno proverljiv: svaki učesnik može da rekonstruiše izračunavanje finalne vrednosti i potvrdi izbor dobitnika.

Bodovanje

- Pametni ugovor i deployment na Sepolia mreži: 9 poena
- Web3/React integracija i front-end aplikacija: 7 poena
- Testiranje funkcionalnosti — obavezni unit testovi (Truffle/Hardhat ili Remix) uz end-to-end proveru nad ugovorom na Sepolia mreži: 4 poena

Ukupno: 20 poena (bez testiranja: maksimalno 16 poena)

Implementacija se može uraditi u Remix IDE ili u Truffle/Hardhat okruženju. Izbor alata ne utiče na maksimalan broj poena. Bez obzira na izabrani alat, ugovor mora biti deploy-ovan na Sepolia testnu mrežu, a aplikacija (Web3/React) mora biti povezana sa pametnim ugovorom upravo na Sepolia mreži. Time se stiče iskustvo u izradi kompletnih decentralizovanih aplikacija koje obuhvataju i blockchain i web sloj.

Testiranje funkcionalnosti (4 poena) je obavezno i obuhvata dva dela. Prvo, student je dužan da napiše unit testove za pametni ugovor, automatizovane testove u Truffle/Hardhat okruženju ili pomoću Solidity Unit Testing plugin-a u Remix-u koji pokrivaju ključne funkcije i granične slučajeve (validne i nevalidne scenarije, kontrolu pristupa, emitovanje događaja). Drugo, ispravan rad mora biti potvrđen i end-to-end proverom kroz samu aplikaciju (Web3/React) nad ugovorom deploy-ovanim na Sepolia mreži. Ukoliko student ne uradi unit testove, maksimalan broj poena za ceo zadatak iznosi 16 poena (9 + 7). Ovladavanje unit testovima i deployment skriptama predstavlja standardnu praksu u profesionalnom razvoju i čini studente konkurentnijim na tržištu rada.

Projektni zadatak 4: DApp za uslovni escrow sa arbitražom

Cilj projekta

Cilj projekta je da studenti razviju decentralizovanu aplikaciju (DApp) koja implementira escrow (deponovanje sredstava kod posrednika) između kupca i prodavca, sa mogućnošću uključivanja arbitra u slučaju spora.

U ovom zadatku je fokus na uslovnom oslobađanju sredstava i razrešavanju sporova bez poverenja u jednu stranu.

Sredstva se zaključavaju u ugovoru i oslobađaju tek kada su ispunjeni dogovoreni uslovi (potvrda kupca), a u slučaju spora odluku donosi unapred dogovoreni arbitar, čime se postiže pravičnost i zaštita obe strane.

Ključne komponente projekta

- Pametni ugovor koji čuva sredstva u escrow-u, evidentira uloge (kupac, prodavac, arbitar), stanje posla i omogućava oslobađanje, povraćaj ili arbitražnu odluku.
- Front-end aplikacija preko koje kupac kreira i finansira escrow, potvrđuje prijem, pokreće spor, a prodavac i arbitar prate i učestvuju u procesu.
- Integracija sa Web3/ethers.js radi interakcije sa ugovorom i potpisivanja akcija.
- Deployment i testiranje u Truffle ili Hardhat okruženju. Testovi treba da pokriju kreiranje escrow-a, uspešno oslobađanje sredstava prodavcu, povraćaj kupcu, kao i razrešenje spora od strane arbitra u korist svake od strana.

Detalji pametnog ugovora

- Svaki escrow treba da sadrži podatke o kupcu, prodavcu, arbitru, iznosu i trenutnom stanju (kreiran, finansiran, završen, u sporu, razrešen).
- Ugovor treba da podrži tri ishoda: oslobađanje sredstava prodavcu (potvrda kupca), povraćaj kupcu i arbitražnu odluku u slučaju spora.
- Samo ovlašćene uloge mogu pokretati odgovarajuće akcije (npr. samo kupac potvrđuje prijem, samo arbitar razrešava spor).
- Ugovor emituje događaje pri kreiranju, finansiranju, oslobađanju, pokretanju spora i arbitražnoj odluci.

Dodatni kriptografski i bezbednosni zahtevi

- Autentičnost akcija obezbeđuje se kroz kontrolu uloga i ECDSA potpise naloga (MetaMask); samo ovlašćene adrese mogu izvršiti odgovarajuće operacije.
- Sistem mora sprečiti dvostruko oslobađanje istih sredstava i obezbediti reentrancy zaštitu pri svakom transferu.
- Sve promene stanja i transferi moraju biti vremenski obeleženi i javno proverljivi (revizorski trag).
- Potrebno je validirati ulazne parametre i sprečiti da arbitar ili bilo koja strana zaobiđe dogovorena pravila procesa.

Bodovanje

- Pametni ugovor i deployment na Sepolia mreži: 9 poena
- Web3/React integracija i front-end aplikacija: 7 poena

- Testiranje funkcionalnosti - obavezni unit testovi (Truffle/Hardhat ili Remix) uz end-to-end proveru nad ugovorom na Sepolia mreži: 4 poena

Ukupno: 20 poena (bez testiranja: maksimalno 16 poena)

Implementacija se može uraditi u Remix IDE ili u Truffle/Hardhat okruženju. Izbor alata ne utiče na maksimalan broj poena. Bez obzira na izabrani alat, ugovor mora biti deploy-ovan na Sepolia testnu mrežu, a aplikacija (Web3/React) mora biti povezana sa pametnim ugovorom upravo na Sepolia mreži. Time se stiče iskustvo u izradi kompletnih decentralizovanih aplikacija koje obuhvataju i blockchain i web sloj.

Testiranje funkcionalnosti (4 poena) je obavezno i obuhvata dva dela. Prvo, student je dužan da napiše unit testove za pametni ugovor, automatizovane testove u Truffle/Hardhat okruženju ili pomoću Solidity Unit Testing plugin-a u Remix-u koji pokrivaju ključne funkcije i granične slučajeve (validne i nevalidne scenarije, kontrolu pristupa, emitovanje događaja). Drugo, ispravan rad mora biti potvrđen i end-to-end proverom kroz samu aplikaciju (Web3/React) nad ugovorom deploy-ovanim na Sepolia mreži. Ukoliko student ne uradi unit testove, maksimalan broj poena za ceo zadatak iznosi 16 poena (9 + 7). Ovladavanje unit testovima i deployment skriptama predstavlja standardnu praksu u profesionalnom razvoju i čini studente konkurentnijim na tržištu rada.

Projektni zadatak 5: DApp za atomic swap dva ERC-20 tokena (razmena bez poverenja)

Cilj projekta

Cilj projekta je da studenti razviju decentralizovanu aplikaciju (DApp) koja omogućava atomic swap (atomsku razmenu) dva ERC-20 tokena između dve strane, tako da se razmena ili izvrši u potpunosti za obe strane, ili se ne izvrši uopšte.

U ovom zadatku je fokus na razmeni vrednosti bez poverenja u posrednika (trustless exchange).

Korišćenjem mehanizma zasnovanog na hash-locku i time-locku (HTLC princip primenjen na tokene), strane razmenjuju sredstva bez rizika da jedna ostane bez tokena, jer otkrivanje tajne istovremeno otključava obe strane razmene.

Ključne komponente projekta

- Pametni ugovor koji zaključava tokene jedne strane uz hash tajne vrednosti i vremenski rok, a oslobađa ih drugoj strani po otkrivanju tačne tajne, odnosno vraća ih pošiljaocu po isteku roka.
- Front-end aplikacija koja omogućava kreiranje ponude za razmenu, zaključavanje tokena, otkrivanje tajne radi preuzimanja i povraćaj sredstava po isteku roka.
- Integracija sa Web3/ethers.js i podrška za standardno odobravanje (approve) ERC-20 tokena pre zaključavanja.
- Deployment i testiranje u Truffle ili Hardhat okruženju. Testovi treba da pokriju kreiranje zaključanog depozita, validno otključavanje tačnom tajnom, nevalidne pokušaje i povraćaj sredstava nakon isteka roka.

Detalji pametnog ugovora

- Svaki zaključani depozit treba da sadrži pošiljaoca, primaoca, adresu tokena, iznos, kriptografski otisak tajne (hashlock) i vremenski rok (timelock).
- Ugovor obezbeđuje dve operacije: oslobađanje tokena otkrivanjem validne tajne i povraćaj tokena pošiljaocu nakon isteka roka.
- Sistem mora onemogućiti višestruko otključavanje ili istovremeno preuzimanje i povraćaj istih sredstava.
- Ugovor emituje događaje pri kreiranju depozita, uspešnom otključavanju (uz otkrivanje tajne) i povraćaju sredstava.

Dodatni kriptografski i bezbednosni zahtevi

- Za otključavanje se koristi hash funkcija (npr. keccak256 ili SHA-256) kojom se otkrivena tajna upoređuje sa unapred zabeleženim otiskom (hashlock).
- Rok trajanja meri se vremenskim oznakama bloka (block timestamp) radi preciznog povraćaja po isteku.
- Neophodno je obezbediti reentrancy zaštitu i bezbedno rukovanje ERC-20 transferima (provera povratnih vrednosti / safeTransfer).
- Sistem treba da omogući transparentnost: svaka strana može da potvrdi stanje depozita i validnost razmene.

Bodovanje

- Pametni ugovor i deployment na Sepolia mreži: 9 poena

- Web3/React integracija i front-end aplikacija: 7 poena
- Testiranje funkcionalnosti - obavezni unit testovi (Truffle/Hardhat ili Remix) uz end-to-end proveru nad ugovorom na Sepolia mreži: 4 poena

Ukupno: 20 poena (bez testiranja: maksimalno 16 poena)

Implementacija se može uraditi u Remix IDE ili u Truffle/Hardhat okruženju. Izbor alata ne utiče na maksimalan broj poena. Bez obzira na izabrani alat, ugovor mora biti deploy-ovan na Sepolia testnu mrežu, a aplikacija (Web3/React) mora biti povezana sa pametnim ugovorom upravo na Sepolia mreži. Time se stiče iskustvo u izradi kompletnih decentralizovanih aplikacija koje obuhvataju i blockchain i web sloj.

Testiranje funkcionalnosti (4 poena) je obavezno i obuhvata dva dela. Prvo, student je dužan da napiše unit testove za pametni ugovor, automatizovane testove u Truffle/Hardhat okruženju ili pomoću Solidity Unit Testing plugin-a u Remix-u koji pokrivaju ključne funkcije i granične slučajeve (validne i nevalidne scenarije, kontrolu pristupa, emitovanje događaja). Drugo, ispravan rad mora biti potvrđen i end-to-end proverom kroz samu aplikaciju (Web3/React) nad ugovorom deploy-ovanim na Sepolia mreži. Ukoliko student ne uradi unit testove, maksimalan broj poena za ceo zadatak iznosi 16 poena (9 + 7). Ovladavanje unit testovima i deployment skriptama predstavlja standardnu praksu u profesionalnom razvoju i čini studente konkurentnijim na tržištu rada.

Projektni zadatak 6: DApp za token-gated enkriptovani pristup sadržaju

Cilj projekta

Cilj projekta je da studenti razviju decentralizovanu aplikaciju (DApp) koja omogućava pristup enkriptovanom sadržaju samo korisnicima koji poseduju odgovarajući token ili NFT (token-gated access), pri čemu se ključ za dekripciju isporučuje isključivo ovlašćenim vlasnicima.

U ovom zadatku je fokus na kontroli pristupa zasnovanoj na vlasništvu nad tokenom uz poverljivost sadržaja.

Sadržaj se čuva enkriptovano van blockchain-a (npr. IPFS), dok ugovor služi za proveru vlasništva nad tokenom i za kontrolisanu distribuciju enkriptovanog simetričnog ključa onima koji ispunjavaju uslov.

Ključne komponente projekta

- Pametni ugovor koji evidentira sadržaje (hash, lokacija, uslov pristupa - potreban token/NFT) i proverava da li određena adresa poseduje traženi token pre nego što dobije pravo na ključ.
- Front-end aplikacija koja omogućava vlasniku sadržaja da ga enkriptuje i objavi uslov pristupa, a korisniku da dokaže vlasništvo nad tokenom i preuzme/dekriptuje sadržaj.
- Integracija sa Web3/ethers.js radi provere vlasništva (npr. `balanceOf` / `ownerOf`) i interakcije sa ugovorom.
- Deployment i testiranje u Truffle ili Hardhat okruženju. Testovi treba da pokriju objavu sadržaja, uspešan pristup vlasnika tokena, odbijanje korisnika bez tokena i proveru integriteta sadržaja.

Detalji pametnog ugovora

- Ugovor treba da beleži metapodatke sadržaja: identifikator, hash, spoljnu lokaciju (IPFS) i uslov pristupa (adresa token ugovora i potrebna količina ili konkretan NFT).
- Potrebno je omogućiti proveru da li adresa zadovoljava uslov pristupa u trenutku zahteva.
- Enkriptovani simetrični ključ se ovlašćenom korisniku isporučuje enkriptovan njegovim javnim ključem (ili po pristupnom mehanizmu), tako da samo on može da ga dekriptuje.
- Ugovor emituje događaje pri objavi sadržaja, odobravanju pristupa i opozivu prava pristupa.

Dodatni kriptografski i bezbednosni zahtevi

- Sadržaj se enkriptuje simetričnim algoritmom (AES-256-GCM), dok se simetrični ključ deli ovlašćenom korisniku asimetričnom enkripcijom (RSA ili ECC) njegovim javnim ključem.
- Integritet sadržaja proverava se hash funkcijom (npr. SHA-256), a otisak se beleži na blockchain-u.
- Privatni podaci, sadržaj i simetrični ključ ne smeju se upisivati na blockchain; čuvaju se isključivo hash-evi i metapodaci.
- Pri simetričnoj enkripciji obezbediti jedinstvenost inicijalizacionih vektora (IV) radi zaštite od kriptanalize.

Bodovanje

- Pametni ugovor i deployment na Sepolia mreži: 9 poena

- Web3/React integracija i front-end aplikacija: 7 poena
- Testiranje funkcionalnosti - obavezni unit testovi (Truffle/Hardhat ili Remix) uz end-to-end proveru nad ugovorom na Sepolia mreži: 4 poena

Ukupno: 20 poena (bez testiranja: maksimalno 16 poena)

Implementacija se može uraditi u Remix IDE ili u Truffle/Hardhat okruženju. Izbor alata ne utiče na maksimalan broj poena. Bez obzira na izabrani alat, ugovor mora biti deploy-ovan na Sepolia testnu mrežu, a aplikacija (Web3/React) mora biti povezana sa pametnim ugovorom upravo na Sepolia mreži. Time se stiče iskustvo u izradi kompletnih decentralizovanih aplikacija koje obuhvataju i blockchain i web sloj.

Testiranje funkcionalnosti (4 poena) je obavezno i obuhvata dva dela. Prvo, student je dužan da napiše unit testove za pametni ugovor, automatizovane testove u Truffle/Hardhat okruženju ili pomoću Solidity Unit Testing plugin-a u Remix-u koji pokrivaju ključne funkcije i granične slučajeve (validne i nevalidne scenarije, kontrolu pristupa, emitovanje događaja). Drugo, ispravan rad mora biti potvrđen i end-to-end proverom kroz samu aplikaciju (Web3/React) nad ugovorom deploy-ovanim na Sepolia mreži. Ukoliko student ne uradi unit testove, maksimalan broj poena za ceo zadatak iznosi 16 poena (9 + 7). Ovladavanje unit testovima i deployment skriptama predstavlja standardnu praksu u profesionalnom razvoju i čini studente konkurentnijim na tržištu rada.

Projektni zadatak 7: DApp za prag (threshold) enkripciju i distribuirano dešifrovanje

Cilj projekta

Cilj projekta je da studenti razviju decentralizovanu aplikaciju (DApp) koja omogućava prag-enkripciju (threshold encryption), gde se poruka može dekriptovati samo ako sarađuje najmanje M od ukupno N čuvara ključa, pri čemu nijedan pojedinac ne poseduje ceo privatni ključ.

U ovom zadatku je fokus na distribuiranom poverenju i otpornosti na kompromitaciju manjine čuvara.

Time se obezbeđuje da nijedna pojedinačna strana ne može sama da pristupi tajni, a istovremeno sistem ostaje funkcionalan i kada deo čuvara nije dostupan ili je kompromitovan, sve dok ih najmanje M sarađuje.

Ključne komponente projekta

- Pametni ugovor koji beleži metapodatke o zaštićenoj poruci: identifikator, vlasnika, prag M , listu čuvara ključa i njihove potvrde o učešću u dešifrovanju.
- Front-end aplikacija koja omogućava enkripciju poruke, distribuciju delova ključa čuvarima i prikupljanje delimičnih dešifrovanja radi rekonstrukcije kada se dostigne prag M .
- Integracija sa Web3/ethers.js radi koordinacije procesa i evidencije učešća na lancu.
- Deployment i testiranje u Truffle ili Hardhat okruženju. Testovi treba da pokriju registraciju zaštićene poruke, potvrde čuvara, neuspešno dešifrovanje kada učestvuje manje od M čuvara i uspešno dešifrovanje kada se dostigne prag.

Detalji pametnog ugovora

- Ugovor treba da registruje zaštićenu poruku tako što beleži njen identifikator, prag M , listu čuvara i kriptografski otisak (hash) - ne i sam sadržaj.
- Sadržaj poruke i delovi ključa se ne čuvaju na blockchain-u; na lancu se vode samo metapodaci, potvrde učešća i status procesa.
- Ugovor treba da beleži koje su čuvari dostavili svoj delimični doprinos dešifrovanju i da proverava da li je dostignut prag M .
- Ugovor emituje događaje pri registraciji poruke, potvrdi čuvara i kada se dostigne prag potreban za dešifrovanje.

Dodatni kriptografski i bezbednosni zahtevi

- Prag-shema se zasniva na deljenju ključa na N delova uz prag M (npr. Shamir-ovo deljenje tajne ili threshold kriptografija); manje od M delova ne daje nikakvu informaciju o ključu.
- Delovi ključa namenjeni čuvarima moraju biti zaštićeni enkripcijom javnim ključem svakog čuvara, tako da samo on može da dekriptuje svoj deo.
- Sistem mora garantovati da je rekonstrukcija/dešifrovanje moguće isključivo pri dostizanju praga M .
- Potrebno je obezbediti revizorski trag: evidenciju čuvara i vremena njihovog učešća, uz mogućnost provere ispravnosti delimičnih doprinosa.

Bodovanje

- Pametni ugovor i deployment na Sepolia mreži: 9 poena
- Web3/React integracija i front-end aplikacija: 7 poena

- Testiranje funkcionalnosti - obavezni unit testovi (Truffle/Hardhat ili Remix) uz end-to-end proveru nad ugovorom na Sepolia mreži: 4 poena

Ukupno: 20 poena (bez testiranja: maksimalno 16 poena)

Implementacija se može uraditi u Remix IDE ili u Truffle/Hardhat okruženju. Izbor alata ne utiče na maksimalan broj poena. Bez obzira na izabrani alat, ugovor mora biti deploy-ovan na Sepolia testnu mrežu, a aplikacija (Web3/React) mora biti povezana sa pametnim ugovorom upravo na Sepolia mreži. Time se stiče iskustvo u izradi kompletnih decentralizovanih aplikacija koje obuhvataju i blockchain i web sloj.

Testiranje funkcionalnosti (4 poena) je obavezno i obuhvata dva dela. Prvo, student je dužan da napiše unit testove za pametni ugovor, automatizovane testove u Truffle/Hardhat okruženju ili pomoću Solidity Unit Testing plugin-a u Remix-u koji pokrivaju ključne funkcije i granične slučajeve (validne i nevalidne scenarije, kontrolu pristupa, emitovanje događaja). Drugo, ispravan rad mora biti potvrđen i end-to-end proverom kroz samu aplikaciju (Web3/React) nad ugovorom deploy-ovanim na Sepolia mreži. Ukoliko student ne uradi unit testove, maksimalan broj poena za ceo zadatak iznosi 16 poena (9 + 7). Ovladavanje unit testovima i deployment skriptama predstavlja standardnu praksu u profesionalnom razvoju i čini studente konkurentnijim na tržištu rada.

Projektni zadatak 8: DApp za decentralizovani sistem reputacije sa potpisanim atestacijama

Cilj projekta

Cilj projekta je da studenti razviju decentralizovanu aplikaciju (DApp) koja omogućava izgradnju sistema reputacije na osnovu kriptografski potpisanih atestacija (svedočanstava) koje korisnici izdaju jedni drugima, uz proverljivost autentičnosti i sprečavanje falsifikovanja.

U ovom zadatku je fokus na autentičnosti i neporecivosti ocena/svedočanstava.

Svaka atestacija je digitalno potpisana od strane izdavaoca, čime se garantuje da ocena zaista potiče od navedene adrese i da ne može biti naknadno izmenjena, dok ukupna reputacija ostaje transparentno proverljiva.

Ključne komponente projekta

- Pametni ugovor koji evidentira atestacije (izdavalac, subjekt, ocena/tip, vremenska oznaka) i agregira reputaciju subjekta na proverljiv način.
- Front-end aplikacija koja omogućava korisniku da izda potpisanu atestaciju drugom korisniku, pregleda atestacije i prikaže agregiranu reputaciju subjekta.
- Integracija sa Web3/ethers.js radi potpisivanja atestacija (MetaMask) i njihove verifikacije/zapisa.
- Deployment i testiranje u Truffle ili Hardhat okruženju. Testovi treba da pokriju izdavanje validne atestacije, odbijanje nevalidnog potpisa, sprečavanje dvostrukog ocenjivanja istog subjekta od istog izdavaoca i ispravnu agregaciju reputacije.

Detalji pametnog ugovora

- Ugovor treba da beleži atestacije sa podacima o izdavaocu, subjektu, sadržaju ocene (ili njenom otisku) i vremenu izdavanja.
- Potrebno je verifikovati da atestacija nosi validan potpis navedenog izdavaoca pre nego što se prihvati.
- Sistem treba da spreči zloupotrebe poput višestrukog ocenjivanja istog subjekta od strane iste adrese ili samoocenjivanja.
- Ugovor emituje događaje pri izdavanju atestacije, opozivu (ako je predviđen) i promeni agregirane reputacije.

Dodatni kriptografski i bezbednosni zahtevi

- Atestacije se potpisuju asimetrično (ECDSA); verifikacija potpisa vrši se na osnovu javno dostupne adrese izdavaoca, bez izlaganja privatnog ključa.
- Integritet sadržaja atestacije obezbeđuje se hash funkcijom (npr. SHA-256/keccak256), a otisak se vezuje za potpis.
- Svaki zapis mora imati vremensku oznaku i biti javno proverljiv (revizorski trag).
- Sistem treba da bude otporan na replay napade (npr. uključivanjem nonce vrednosti ili konteksta u potpisanu poruku).

Bodovanje

- Pametni ugovor i deployment na Sepolia mreži: 9 poena
- Web3/React integracija i front-end aplikacija: 7 poena

- Testiranje funkcionalnosti - obavezni unit testovi (Truffle/Hardhat ili Remix) uz end-to-end proveru nad ugovorom na Sepolia mreži: 4 poena

Ukupno: 20 poena (bez testiranja: maksimalno 16 poena)

Implementacija se može uraditi u Remix IDE ili u Truffle/Hardhat okruženju. Izbor alata ne utiče na maksimalan broj poena. Bez obzira na izabrani alat, ugovor mora biti deploy-ovan na Sepolia testnu mrežu, a aplikacija (Web3/React) mora biti povezana sa pametnim ugovorom upravo na Sepolia mreži. Time se stiče iskustvo u izradi kompletnih decentralizovanih aplikacija koje obuhvataju i blockchain i web sloj.

Testiranje funkcionalnosti (4 poena) je obavezno i obuhvata dva dela. Prvo, student je dužan da napiše unit testove za pametni ugovor, automatizovane testove u Truffle/Hardhat okruženju ili pomoću Solidity Unit Testing plugin-a u Remix-u koji pokrivaju ključne funkcije i granične slučajeve (validne i nevalidne scenarije, kontrolu pristupa, emitovanje događaja). Drugo, ispravan rad mora biti potvrđen i end-to-end proverom kroz samu aplikaciju (Web3/React) nad ugovorom deploy-ovanim na Sepolia mreži. Ukoliko student ne uradi unit testove, maksimalan broj poena za ceo zadatak iznosi 16 poena (9 + 7). Ovladavanje unit testovima i deployment skriptama predstavlja standardnu praksu u profesionalnom razvoju i čini studente konkurentnijim na tržištu rada.

Projektni zadatak 9: DApp za verifikabilne kredencijale i decentralizovani identitet (DID) sa selektivnim otkrivanjem

Cilj projekta

Cilj projekta je da studenti razviju decentralizovanu aplikaciju (DApp) koja omogućava izdavanje, čuvanje i verifikaciju verifikabilnih kredencijala (Verifiable Credentials) vezanih za decentralizovani identitet (DID), uz mogućnost selektivnog otkrivanja samo onih atributa koji su zaista potrebni.

U ovom zadatku je fokus na samostalnom upravljanju identitetom (self-sovereign identity) i privatnosti.

Korisnik (holder) prima kredencijal od izdavaoca, čuva ga kod sebe, i prilikom verifikacije otkriva proverilac samo neophodne attribute (npr. da je punoletan, bez otkrivanja datuma rođenja), dok blockchain služi za proveru izdavaoca i status kredencijala.

Ključne komponente projekta

- Pametni ugovor koji vodi registar izdavalaca i status kredencijala (važeći/opozvan), uz otiske kredencijala i vremenske oznake, ali bez ličnih podataka.
- Front-end aplikacija u tri uloge: izdavalac (izdaje potpisan kredencijal), holder (čuva i pravi prezentaciju sa selektivnim otkrivanjem) i proverilac (verifikuje potpis, izdavaoca i status).
- Integracija sa Web3/ethers.js radi potpisivanja kredencijala, provere izdavaoca i statusa opoziva na lancu.
- Deployment i testiranje u Truffle ili Hardhat okruženju. Testovi treba da pokriju izdavanje kredencijala, verifikaciju validnog kredencijala, selektivno otkrivanje atributa, odbijanje opozvanog kredencijala i odbijanje nevalidnog potpisa.

Detalji pametnog ugovora

- Ugovor treba da beleži registar ovlašćenih izdavalaca i da omogući opoziv kredencijala (npr. preko liste opoziva ili statusa po identifikatoru kredencijala).
- Na lancu se čuvaju samo otisci (hash) i status kredencijala, kao i metapodaci o izdavaocu - nikada lični podaci.
- Potrebno je omogućiti svakom proverilac da utvrdi da li je izdavalac ovlašćen i da li je kredencijal i dalje važeći.
- Ugovor emituje događaje pri registraciji izdavaoca, izdavanju kredencijala (otisak) i opozivu.

Dodatni kriptografski i bezbednosni zahtevi

- Kredencijali se digitalno potpisuju asimetričnom kriptografijom (ECDSA); verifikacija se vrši na osnovu javnog ključa/adrese izdavaoca.
- Selektivno otkrivanje treba realizovati kriptografskim mehanizmom (npr. Merkle stablo atributa sa dokazima pripadnosti ili ZK pristup), tako da se otkrivaju samo traženi atributi.
- Lični podaci se ne upisuju na blockchain; koriste se isključivo otisci, status i metapodaci.
- Sistem mora biti otporan na replay i na ponovno korišćenje prezentacije (npr. uključivanjem nonce/domena u potpisanu prezentaciju), uz jasan prikaz svrhe verifikacije korisniku.

Bodovanje

- Pametni ugovor i deployment na Sepolia mreži: 9 poena

- Web3/React integracija i front-end aplikacija: 7 poena
- Testiranje funkcionalnosti - obavezni unit testovi (Truffle/Hardhat ili Remix) uz end-to-end proveru nad ugovorom na Sepolia mreži: 4 poena

Ukupno: 20 poena (bez testiranja: maksimalno 16 poena)

Implementacija se može uraditi u Remix IDE ili u Truffle/Hardhat okruženju. Izbor alata ne utiče na maksimalan broj poena. Bez obzira na izabrani alat, ugovor mora biti deploy-ovan na Sepolia testnu mrežu, a aplikacija (Web3/React) mora biti povezana sa pametnim ugovorom upravo na Sepolia mreži. Time se stiče iskustvo u izradi kompletnih decentralizovanih aplikacija koje obuhvataju i blockchain i web sloj.

Testiranje funkcionalnosti (4 poena) je obavezno i obuhvata dva dela. Prvo, student je dužan da napiše unit testove za pametni ugovor, automatizovane testove u Truffle/Hardhat okruženju ili pomoću Solidity Unit Testing plugin-a u Remix-u koji pokrivaju ključne funkcije i granične slučajeve (validne i nevalidne scenarije, kontrolu pristupa, emitovanje događaja). Drugo, ispravan rad mora biti potvrđen i end-to-end proverom kroz samu aplikaciju (Web3/React) nad ugovorom deploy-ovanim na Sepolia mreži. Ukoliko student ne uradi unit testove, maksimalan broj poena za ceo zadatak iznosi 16 poena (9 + 7). Ovladavanje unit testovima i deployment skriptama predstavlja standardnu praksu u profesionalnom razvoju i čini studente konkurentnijim na tržištu rada.